

Error Handling

In languages like **JS**, errors are handled by "**throwing**" exceptions and catching them in bulky **try/catch** blocks. Go rejects this design. Instead, treats errors as normal, returnable values using a built-in interface.

1. The Built-in Error Interface

Under the hood, Go has a globally available, pre-defined interface called **error**.

```
type error interface {  
    Error() string  
}
```

Any custom struct you create that has an **Error()** method returning a **string** automatically satisfies this **interface** and can be treated as a standard Go error!

2. The Standard Return Pattern

Because Go functions can return multiple values, the universal standard is to return the expected data first, and the error interface second.

- Example: **f, err := os.Open("data.txt")**
- If the file opens successfully, **f** contains the file data, and **err** will be exactly **nil (empty)**.
- If the file does not exist, **f** will be empty, and **err** will contain the error interface with the failure details.

3. Checking and Handling Errors (if err != nil)

Because errors are just values, you must explicitly check them immediately after calling a function that might fail.

```
f, err := os.Open("data.txt")  
if err != nil {  
    // The operation failed! Handle it here.  
    fmt.Println(err)  
    return  
} // If we reach here, it is 100% safe to use 'f'
```

The Println Magic: When you pass the **err** variable into **fmt.Println(err)**, the **fmt** package detects that it satisfies the error interface and automatically calls its **.Error()** method to print the human-readable string to the console.